プログラミング言語実験-第 4 回-

2412747　江尾祐哉

1 問目

・コンパイルおよび実行方法

「g++ 2412747_4-1.cpp -o 2412747_4-1」とコンパイルし、「./2412747_4-1　a b c」のように a と b と c にそれぞれ数値を入力して実行する。

・実装概要

コマンドライン引数で入力された 3 種類の値に対して、

$$ax^2 + bx + c = 0$$

の 2 次方程式を解くという課題である。2 次方程式の解の公式は、

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

であるので、入力された値をこれに代入してプログラムに計算させれば解が導き出されるのだが、ここで注意することがある。それは入力によっては虚数解が期待される場合である。これにより特に対策をせずに計算をさせた場合、それぞれの値を double 型で持つとすると、xが NaN となり導き出せない。そのため、$b^2 - 4ac$が 0 より大きいかどうかをあらかじめ調べて、それによって計算の方法を場合わけさせなければならない。この点を踏まえてコードの説明をする。

まずは引数のチェックを行った。引数が 3 つでない場合を弾き、また、check_arg 関数によって、入力されたものが数字列であるか否かを判定した。今回は計算に必要な a、b、c を、構造体を利用して保持させる必要があった。そのためそれらと、それから方程式から導き出せる 2 つの解を保持する t_quad 構造体を用意した。また、解は実部と虚部を含む可能性があるため、t_comp_num 型を定義してそれを利用した。用意した構造体のメンバに入力された値を atoi 関数で格納する initialize_quad 関数で初期設定を行った後、a が 0 でないかを調べて、二次方程式を計算する calc_quad 関数を実行

させた。

この関数では二次方程式の計算をするわけだが、先述したように虚数解かどうかで場合わけさせて計算している。今回は課題 PDF で解と係数の関係を利用するよう記載されているので、和を表す sum 変数と積を表す pro 変数、そして判別式を表す disc 変数を用意した。解と係数の関係とは、2 つの解について、

$$\alpha + \beta = -\frac{b}{a}, \quad \alpha\beta = \frac{c}{a}$$

が成り立つことなので、これに当てはめて計算を行った。まずは判別式が 0 以上、すなわち虚数解でない場合についてだが、解と係数の関係単体では解を求めることができないため、1 つの解については解の公式を用いて普通に計算を行う。この後もう一方の解を、0 以外の場合は積を、0 の場合のみ和を利用して求めている。0 での場合分けは 0 除算回避のためだが、それならすべての場合で和を利用して差分を取れば良いということが思いつく。しかしながら、double 型では近い値の引き算の精度が失われる、桁落ちが発生するようになっている。これを回避するためにできるだけ積の割り算を使用するようにしている。これによって求まったら、t_comp_num 型の値に実数部は解を、虚数部は 0 を代入して終了となる。

次は判別式が 0 未満、すなわち虚数解である場合についてである。こちらも先ほどと同様に、解と係数の関係単体では求められないので、1 つの解を求めている。こちらは 0 である可能性がないので積の割り算を利用して求めるが、分母に虚数が存在するためにそのままでは求められない。そこで denom 変数によって共役複素数をかけて分母を実数にする処理を行い、計算を可能にしている。あとはこれによって値を代入して完了となる。なお、解の公式だけではなく、解と係数の関係を利用して求めさせたのは、実数解において先ほど述べた桁落ちを回避できる点にあると考えている。足し算や引き算の計算をできるだけ抑えることで、これを可能にしているという理由によりそのように考えた。しかし虚数解では、2 つの解が共役複素数となるためあまり効果はないと感じている。

以上により解がもとまったためあとは出力するだけなのだが、ここで print_quad 関数という関数を実装して出力を工夫した。全部を説明すると冗長になるので割愛するが、例えば b が 0 のときにわざわざ$0x$と出力したり、解が負のときに+-2 のように符号が重複したりすることを防ぐためである。また重解であるときは重解だと出力するようにした。これらの詳細はコードにコメントアウトして載せてある。これにより滑らかな出力が可能となり、ユーザーが見やすいようになった。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 3 -6 3
3x² - 6x + 3 = 0の解は、
x = 1 (重解)です。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 5 6 2
5x² + 6x + 2 = 0の解は、
x = -0.6 - 0.2i (重解)です。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 2 9 -5
2x² + 9x - 5 = 0の解は、
x = -5と、x = 0.5です。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 3 0 0
3x²  = 0の解は、
x = 0 (重解)です。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 3 0 64
3x² + 64 = 0の解は、
x = -4.6188i (重解)です。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1
引数の数が違います。a、b、cの値を入力して実行してください。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 0 9 8
aが0であるのは不正です。0以外の数値を入力してください。
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-1 -4 74 37
-4x² + 74x + 37 = 0の解は、
x = 18.9872と、x = -0.487171です。
```

図1　1問目の実行結果

2 問目

・コンパイルおよび実行方法

「g++ 2412747_4-2.cpp -o 2412747_4-2」とコンパイルし、「./2412747_4-2」のように実行する。

・実装概要

いよいよこの問題から C++の真骨頂であるクラスの利用が始まる。この問題はクラスの基本的な構文、子クラスへの継承、オーバーライドを実際に記述していく課題である。用意するクラスは教室というクラスと、これを継承した PC 教室というクラスで、それぞれのクラスのオブジェクトの情報を出力することがこのプログラムの内容である。従って main 関数ではクラスをインスタンス化して、それぞれの情報を出力するメンバ関数を実行すれば良い。よって、public にするのはコンストラクタ及び出力用関数の PrintMem 関数のみでよく、クラスの情報を記憶するメンバ変数はすべて private とすれば良い。ただし今回は親クラスの ClassRoom のメンバは子クラスの PCRoom に継承

されるので、private なメンバは protected にしなくてはならない。また PrintMem 関数と PrintMem に使用される PrintProjectorInfo 関数は子クラスでオーバーライドするので、仮想関数とする必要がある。このようにしてクラス及びメンバを定義した。

親クラスの ClassRoom ではコンストラクタで試験定員、座席定員、座席定員(コロナ禍)、プロジェクターの有無、スクリーンの有無を初期化した。ここで定員の値は可読性の観点から const 定数として定義した。プロジェクターやスクリーンの有無は bool 型で直接定義した。PCRoom クラスでは親クラスの ClassRoom コンストラクタにより初期化したのち、未初期化の PC の台数、プロジェクターの台数、マイクの有無、無線 LAN の有無を初期化した。ここで、親クラスのオブジェクトにプロジェクターが存在しないときは、継承という観点からプロジェクターの台数を 0 台にするように実装した。どのみち PrintProjectorInfo でプロジェクターの有無によって出力を変えるように実装しているため MUST ではないが、プロジェクターが存在しないのに台数が 0 でないのは不自然であるため、そのようにした。

次に出力用の PrintMem 関数について説明する。基本的には適切なメッセージをつけて、オブジェクトの情報を出力するようにしている。子クラスではこれをオーバーライドする形で使用しており、新たに子クラス限定の情報を出力するようにしている。PrintProjectorInfo 関数は、プロジェクターの情報が親クラスであるかないかを、子クラスでは台数を出力しないといけなく、内容が統一されていないために個別に処理を行うため実装した。これらの実装により正常な実行が可能となった。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-2
この教室には、
座席が150席あり、試験時には100席だけ使用できます。また、コロナ禍での座席定員は50
席としていました。
この教室にはプロジェクターがあり、スクリーンはあります。
この教室には、
座席が150席あり、試験時には100席だけ使用できます。また、コロナ禍での座席定員は50
席としていました。
この教室にはプロジェクターが4台あり、スクリーンはあります。
PCの台数は150台で、マイクはあります。
また、無線LANが整備されています。
```

図2　2問目の実行結果

3 問目

・コンパイルおよび実行方法

「g++ 2412747_4-3.cpp -o 2412747_4-3」とコンパイルし、「./2412747_4-3」のように実行する。

・実装概要

この問題は、クラスを利用して行列の様々な演算を行うというものである。課題では行列クラスに縦と横のサイズを格納するメンバ変数、行列として定義するロジック、オブジェクトを作成するコンストラクタ、和や差、積、行列式を演算するメンバ関数の実装を要求されている。従ってこれを満たすように実装を行なった。

行列に関する全体のクラスを Matrix クラスとして定義した。メンバ変数として行列のサイズを表す width と height 変数、そして行列の値のデータを保持する data 変数を用意した。メンバ関数は、行列の内容を作成するコンストラクタ、オブジェクト破棄時のデストラクタ、行列演算用の add、sub、mul、det 関数、行列式計算で用いる余因子展開の getCofactor 関数、そして出力用の print 関数を用意した。それぞれのメンバ関数について順番に説明をしていく。

まずコンストラクタとは、クラスのインスタンス化が行われた際に自動で実行される関数で、クラス名と同名のものとなっている。ここではまず width と height を引数の w と h で初期化し、data を後のメモリエラー対策のために nullptr で初期化する。次に data を動的にメモリの確保し、引数の 1 次元配列の elem の値を参照して 2 次元配列となる data にそれぞれ値を入れていく。ここで今回はメモリの扱いに十分注意しなければならないことに注意した。というのもコンストラクタの実行の途中でメモリの確保が失敗したとき、それまでに確保したメモリは適切に解放しなければならない。また、1 つの行列をインスタンス化し、もう 1 つの行列をインスタンス化する際に途中でメモリエラーが発生した場合も適切にメモリを解放しなければならない。そのためメモリの確保時に try-catch 文を使用して、メモリエラーが発生したときに投げられる bad_alloc の例外を拾い、それにより失敗時にはメモリを解放するようにしてある。これはコンストラクタ、add、sub、mul、getCofactor、main で行なっている。また、オブジェクトが破棄される際に実行されるデストラクタと呼ばれるものでは、プログラムの終了時に全てのメモリを解放するようにしてある。

次に演算用の関数について説明するが、その前に各演算の方法について整理する。和算と引き算は 2 つの行列の成分を足し合わせたり引いたりするだけで良い。そ

のため 2 つの行列のサイズが等しいことが計算をすることの条件となる。積は、2 つの行列 A、B について、A の第 m 行の各成分と B の第 n 行の各成分の積の和が行列 AB の m 行 n 列の成分となるので、これを演算して求めれば良い。したがって、A の列数と B の行数が等しいことが条件となる。行列式は行数と列数の等しい正方行列において、

$$\sum \mathrm{sgn}\begin{pmatrix} 1 & \cdots & n \\ \vdots & \ddots & \vdots \\ i_1 & \cdots & i_n \end{pmatrix} a_{1i_1} \cdots a_{ni_n}$$

で計算できる式である。例えば 3 次の正方行列、

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

があったときに、

$$\det|A| = 1 \times 1 \times \begin{vmatrix} 5 & 6 \\ 8 & 9 \end{vmatrix} + (-1) \times 2 \times \begin{vmatrix} 4 & 6 \\ 7 & 9 \end{vmatrix} + 1 \times 3 \times \begin{vmatrix} 4 & 5 \\ 7 & 8 \end{vmatrix}$$

$$= (5 \times 9 - 6 \times 8) - 2 \times (4 \times 9 - 6 \times 7) + 3 \times (4 \times 8 - 5 \times 7)$$

$$= 0$$

で表されることとなる。このようにして計算すれば良い。

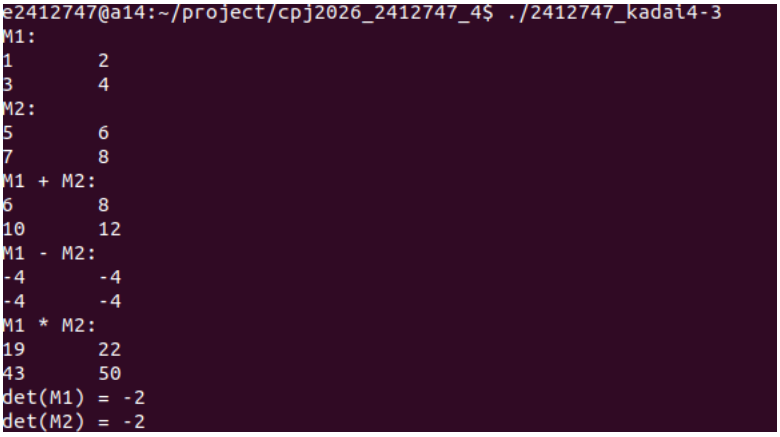
それぞれの計算ロジックは異なるが関数の大枠は同一のものとなっている。まずは演算を行う上で、行列のサイズが正しいかどうかを判別し、不適当であれば invalid_argument の例外を投げるようにした。このチェックの終了後、次は行列の成分数に合わせた int 型の 1 次元配列 elem を用意し、それぞれの要素に行列の成分の演算結果を入れていった。ここでも動的にメモリを確保しているため、メモリエラーを拾うようにしている。演算結果を全て格納したあとは行列クラスのオブジェクトを生成し、コンストラクタの実行後は不要になった elem を解放して、オブジェクトを返すようにした。これにより演算部分の実装が完了した。ここで行列式の演算には余因子展開を用いるように指定されているが、これは任意の大きさの行列式を計算できるように、3 次行列式の計算に用いられるようなサラスの式を使用しないという意味であると解釈した。そのため、先ほど示した計算ロジックに基づいて実装すれば特に問題はない。

行列および行列の演算結果の出力は行列クラスの print 関数で行なっている。ここ

では行列の各成分を出力するが、出力を見やすくするためにタブを使用して区切っている。

最後に main 関数だが、まずは任意の大きさの配列を 2 つ用意するようにした。これは行列の成分値を表すもので、これは課題 PDF でのプログラム内で成分の値を指定するという指示に沿ったものとなっている。次に行列の大きさを表す変数を 2 つの行列についてそれぞれ縦と横の分だけ用意し、数値を指定した。この次に最初に作成した配列とこの大きさが等しいかどうかを判別し、不正であったらエラーを出すようにしている。これは行列クラスのコンストラクタで不正な配列が作成されることを防ぐためである。その後、2 つの行列クラスのオブジェクトの宣言を行い、それぞれの演算を行なった。ここでは try-catch 文を使用することで、行列クラスのコンストラクタおよびメンバ関数内で投げられた例外をしっかりと拾って、エラーメッセージとともに異常終了をすることを可能にしている。これにより実装が完了した。

・実行結果



```
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-3
M1:
1       2
3       4
M2:
5       6
7       8
M1 + M2:
6       8
10      12
M1 - M2:
-4      -4
-4      -4
M1 * M2:
19      22
43      50
det(M1) = -2
det(M2) = -2
```

図3　3問目の実行結果

4 問目

・コンパイルおよび実行方法

「g++ 2412747_4-4.cpp -o 2412747_4-4」とコンパイルし、「./2412747_4-4

alphabetA_2026.txt alphabetB_2026.txt」のように、2 つの引数にそれぞれ単語の長さごとの出現頻度を調べたいテキストファイルのパス名を指定して実行する。

・実装概要

この課題では、コマンドライン引数に与えられた、英文の記載された 2 つのテキストファイルから単語の長さ毎の出現頻度を読み取るプログラムを実装する。PDF では、単語の長さと出現頻度をクラスで定義してカプセル化することや出現頻度のソートプログラムの実装を要求されている。そのため今回は、出現頻度等を表す WordFreq クラスと、文章解析用の TextAnayzer クラスを用意した。

WordFreq クラスでは、初期化を行うコンストラクタおよびコピーコンストラクタ、単語の長さを返す getLen 関数、単語の出現頻度を返す getFreq 関数、出現頻度をインクリメントする addFreq 関数を定義し、private なメンバ変数として、単語の長さを表す len と単語の出現頻度を表す freq をカプセル化して定義した。これをどのように扱うかは次の TextAnalyzer クラスの説明であわせて行う。

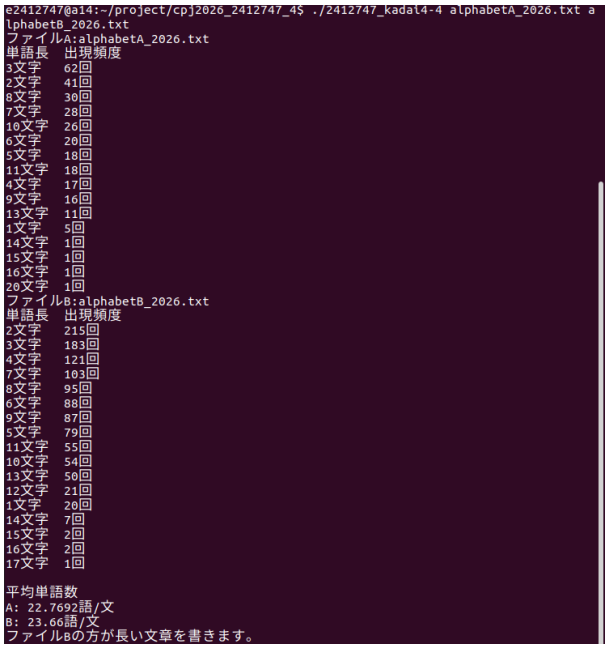
TextAnalyzer クラスでは、メンバ変数として単語の vector の words、出現頻度の vector の freqtable、解析する文を格納する text を定義した。メンバ関数としては、ファイルの読み込みおよび text の構成を行う loadFile、単語の出現頻度の計算をする countFreq、出現頻度の多い順にソートする sortFreq、出力用の print と printCompare、文の数を返す getSentenceCount、総単語数を返す getTotalWord、1 文あたりの平均単語数を返す getWordAve を定義した。まず loadFile では、引数の文字列のファイルを開き、それを 1 行ずつ読み取って空白で区切っていき、区切られた文を text に格納している。このとき実装して色々試して見たところ、元のテキストファイルにはどうやら＼r が含まれていたらしく、そのためこれを除去するように条件を追加した。これを行わないと正常な実行結果が得られなかった。

次に、countFreq で、構成された text を単語毎に区切ってその単語長を測定し、出現頻度毎にまとめていった。単語の vector の words を初期化し、文字列型の tmp を宣言した。1 文字ずつ text から文字を読み取り、その文字の内容をみて、単語が英語かもしくは英語+数字かを判別し、それを小文字に統一する。そして空白が現れたらそこまでが単語であるため、それを words に追加する。この流れを行なってまずは単語群をまとめた。そのあとは単語の最大長を記録する max_len 変数を宣言し、単語群の中から単語の最大長がいくらかを求めた。そしてその値から単語長ごとの出現頻度を

記録するための出現頻度テーブルを構成し、そして再度 for 文により出現頻度を記録していった。これにより出力するものの用意ができた。

課題では出現頻度の多い順に出力することが要求されているため、出現頻度テーブルを自前のバブルソートを行う sortFreq 関数でソートを行い整理した。そして出力用の printFreq 関数で出現頻度が多い順に出力を行なった。これを 1 つ目のテキストファイル A と 2 つ目のテキストファイル B について行い、最後に printCompare 関数を実行した。ここでは両者の 1 文あたりの単語数を計算する getWordAve により平均単語数を求めて、それの値を比較してどちらの方が長い文章を書くかを出力する。平均を求める getWordAve 関数は、総単語数を返す getTotalWord 関数の値を、文の数を返す getCountSentence 関数の値で割ることで平均値を算出している。総合単語数は vector の大きさを見ることで求めることができる。文の数は text の解析をして区切り文字が来たら 1 文というようにカウントすることで求められる。ここでテキストファイルにピリオドが連続している場所があったため、判定がおかしくならないための対策として区切り文字がピリオドである場合のみ別途処理を行うよう工夫した。これにより正常に平均単語数を求めることができた。以上よりそれぞれの単語長毎の出現頻度とその比較の出力が完了し、課題の要件を満たすことができた。

・実行結果



```
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-4 alphabetA_2026.txt a
lphabetB_2026.txt
ファイルA:alphabetA_2026.txt
単語長  出現頻度
3文字   62回
2文字   41回
8文字   30回
7文字   28回
10文字  26回
6文字   20回
5文字   18回
11文字  18回
4文字   17回
9文字   16回
13文字  11回
1文字   5回
14文字  1回
15文字  1回
16文字  1回
20文字  1回
ファイルB:alphabetB_2026.txt
単語長  出現頻度
2文字   215回
3文字   183回
4文字   121回
7文字   103回
8文字   95回
6文字   88回
9文字   87回
5文字   79回
11文字  55回
10文字  54回
13文字  50回
12文字  21回
1文字   20回
14文字  7回
15文字  2回
16文字  2回
17文字  1回

平均単語数
A: 22.7692語/文
B: 23.66語/文
ファイルBの方が長い文章を書きます。
```

図4　4問目の実行結果

5 問目

・コンパイルおよび実行方法

「g++ 2412747_4-5.cpp -o 2412747_4-5」とコンパイルし、「./2412747_4-5 unknownfile2026.csv typefileT2026.csv typefileW2026.csv typefileR2026.csv」のように、1つ目に調べたい csv ファイル、2〜4 つ目に調査に使用するマルウェアの csv ファイルのパス名を指定して実行する。

・実装概要

この課題は PDF に長々と書かれているが、要はファイルから読み込んだ値から最小二乗法によるモデルを作成して、それらと unknown ファイルの値との距離を見て最小であるものを求めるという内容であり、仕組みはそれほど難しくない。クラスは csv ファイルにある特徴量を記録する DataPoint クラスと、マルウェアファミリーに関する様々な計算や出力を行う MalwareFile クラスの 2 つを用意した。DataPoint クラスは特徴量を初期化するコンストラクタとそれぞれの特徴量を返すゲッターのメンバ関数と、特徴量を保持するメンバ変数によって構成されている。MalwareFile クラスは特徴量を記録する vector の data、最小二乗法のモデルに使用する定数の a と b のメンバ変数と、コンストラクタ、ファイルの読み込みと値の格納をする loadFile、最小二乗法を行う fitModel、点と直線の距離を計算する distToLine、平均距離を計算する avgDist、モデルの出力を行う printModel のメンバ関数によって構成されている。DataPoint はすでに説明仕切ったというほどの内容なので、MalwareFile クラスのメンバ関数について順番に説明していく。

ファイルの読み込みと値の格納を行う loadFile 関数だが、まずは引数に与えられた文字列のファイルの読み込みを行う。文字列型の line を用意し、ファイルを 1 行ずつ読み取っていく。そして 1 行をカンマで区切って値をそれぞれ double 型に変換して、それを DataPoint クラスのオブジェクトとして定義する。この流れをファイル全てを読み切るまで行なった。

次に最小二乗法のモデルを作成する fitModel 関数である。最小二乗法とは、データとモデルとなる直線や曲線との誤差の二乗和が最小となるようにモデルを求める方法

であり、これは数値解析での基本的なものである。今回はモデルとデータとの直線距離が最小となるようにしてモデルを求めた。そのため平均と分散、共分散を用いて、

$$a = \frac{S_{yy} - S_{xx} + \sqrt{\left(S_{yy} - S_{xx}\right)^2 - {S_{xy}}^2}}{2S_{xy}}$$

$$b = \bar{Y} - a\bar{X}$$

の式よりモデルとなる直線の式を求めた。従って fitModel 関数では、最初にデータを取得して平均値を求めたあと、分散と共分散を算出し、最後に係数を求めるという処理を行なっている。また、このモデルを出力する printModel 関数も、モデルの可視化のために用意した。

最小二乗法で求まったモデルと与えられたデータの直線距離の平均を avgDist 関数で求めている。DataPoint の vector を定義して unknown のデータを取得し、その各点とモデルとの距離を合計して、最後にデータの数で割って平均を求めるという仕組みである。データが空なことはあまり考えないが、もしデータがない場合はものすごく大きい数として 10 の 10 乗を返すようにしている。点と直線の距離の計算は distToLine 関数で行なっているが、この計算は、

$$d = \frac{|ax_i - y_i + b|}{\sqrt{a^2 + 1}}$$

に基づいて行われている。

MalwareFilek クラスについての説明が完了したため、最後に main 関数を説明していく。まずは引数のエラーチェックを行った。次にマルウェアクラスを 4 つインスタンス化した。ここでクラスの名前の順番は課題 PDF での実行例に従うようにしている。次にそれぞれの引数の文字列とオブジェクトを対応させて loadFile を実行し、エラーが発生した場合は異常終了するようにした。そして typeT、typeW、typeR のモデルを fitModel で作成し、printModel でモデルが何であるかを出力した。それぞれのモデルと unknown のデータとの平均距離を計算し、最後にそれらの平均距離を比較して条件分岐によって出力を行なった。以上より unknown がマルウェアファミリーT に該当することがわかった。

・実行結果

```
e2412747@a14:~/project/cpj2026_2412747_4$ ./2412747_kadai4-5 unknownfile2026.csv
 typefileT2026.csv typefileW2026.csv typefileR2026.csv
T:y = 0.201711x + 11.4273
W:y = 1.05253x + 11.8097
R:y = 3.97043x + 27.8031
それぞれの平均距離は
T: 5.15809
W: 39.1484
R: 67.6733
よってunknownファイルはマルウェアファミリTに該当します。
```

図5　5問目の実行結果